What Survives a Strict Split? An Empirical Audit of Long-Document Screenplay Modelling for IMDb Rating Prediction

Anonymous authors

***Abstract*—Most screenplay-to-rating studies report accuracy on a random split of an archival corpus. We ask how much of that accuracy survives a stricter split. Our corpus joins $5{,}204$ IMSDb screenplays to IMDb metadata; after an identity audit we keep $5{,}164$ records and publish the exclusion ledger. We also find a silent truncation defect: scripts are cut into 256-word windows, but the encoder takes at most 254 WordPiece tokens, so on $1{,}186$ scripts recovered from the project history 98.4% of windows overflow and 31.9% of tokens are lost. Keeping representation and learner fixed, we change only the partition: random k-fold, title-family GroupKFold, and a chronological 2020–2025 holdout. The best system, which adds runtime and release year to the sentence embeddings, scores $R^2 = 0.549$, 0.536 and 0.309 under the three protocols; a text-only model usable before a film exists scores 0.424, 0.411 and 0.267. Release year and decade hold 11.1% of in-sample split gain yet leave holdout MAE exactly unchanged, and trained the same way, sentence embeddings do no better than TF-IDF. The protocol explains more of the spread in reported accuracy than the model does.**

***Index Terms*—data leakage, evaluation protocol, sentence embeddings, long-document NLP, screenplay analysis, temporal generalisation, grouped cross-validation.**

I. Introduction

Screenplays are long. The raw scripts in our corpus run to about $9{,}900$ words on average, while an encoder such as `all-MiniLM-L6-v2` reads no more than 256 tokens at a time. The usual fix, cheap and common, is to slice each script into overlapping windows, embed them one by one, and average the vectors. Less attention goes to whether the evaluation around it yields a number a practitioner should trust. Corpora built from community archives have two quirks that a shuffled split quietly rewards.

Duplication is the first. A sequel borrows the vocabulary and structure of the film before it, and a remake can reuse whole stretches of plot and dialogue; if one sits in training and the other in test, the test film is easier than it looks. Time is the second. Older films in the archive carry higher ratings on average, so a model that knows the release year can explain a good part of the rating from the calendar alone. Once the test films come from years the model never saw, which is exactly where a screenplay-stage predictor operates, that advantage is gone.

Under both sits a smaller, more technical problem. Window length is set in words, truncation happens in subword tokens, and none of the libraries involved warns when the token budget overflows. We therefore audit an existing SBERT-plus-gradient-boosting pipeline rather than propose a new model, and make three contributions.

- **C1: a record-identity protocol and a published exclusion ledger.** Identity is checked against four keys (SHA-256 content digest, IMDb identifier, title×year, and a normalised title-family stem), and every excluded record is accounted for (Section III).
- **C2: a measurement of silent truncation.** We run the encoder's own tokeniser over the pipeline's windows for $1{,}186$ recovered scripts: 98.4% of the 256-word windows exceed the 254-token budget, and 31.9% of WordPiece tokens never reach the encoder (Section IV-B).
- **C3: a three-protocol evaluation with paired inference.** Representation and learner stay fixed and only the partition changes: random k-fold, title-family GroupKFold, and a chronological 2020–2025 holdout. We give bootstrap intervals, paired Wilcoxon tests with effect sizes, a block-permutation analysis of which features still help on future films, and a like-for-like comparison of sentence embeddings against TF-IDF (Sections V and VI).

We are not claiming a new state of the art. Our main result is the distance between $R^2 = 0.549$ and $R^2 = 0.309$ on the same corpus, features and learner, and the further fall to 0.267 once post-production features are removed.

II. Related Work

Screenplays and film outcomes. Scripts were first used as inputs for forecasting return on investment and box-office revenue [1], [2]. Later studies turned to narrative structure: emotional arcs [3], [4], how characters are portrayed [5], MPAA age suitability and content severity [6]–[8], and tags inferred from plot synopses [9]. Closest to rating regression is predicting award nominations from the script [10], which shares its difficulty: the label is a collective judgement that the text explains only in part. Outside the script, forecasters have used production metadata [11], social-media and Wikipedia activity [12]–[15], trailers [16], and the sentiment of reviews or tweets [17], [18]. Predicting ratings or success from tabular metadata is a routine applied exercise [19]–[23], yet papers rarely describe their partitions beyond a random seed. Vogel [24] points out that theatrical results hinge on release strategy, competition and marketing, and none of these appears in a script.

Representations for long documents. Sentence-BERT [25] learns dense sentence embeddings with a Siamese objective;

it is the encoder we audit. Its context window is short, so working at document level means hierarchical attention [26], sparse attention [27], or chunking and pooling. Chunk-and-pool needs no fine-tuning and runs on a CPU, but its pooling is fixed rather than learned and, as Section IV-B shows, its window length is often given in a unit the encoder does not count in.

Leakage and split design. System rankings can flip under a different random split, and a split should test the kind of generalisation a paper claims [28], [29]. Train–test overlap has been quantified for NLP benchmarks [30], [31], a cross-disciplinary survey lists the main forms of leakage [32], and temporal generalisation [33] and corpus documentation [34], [35] each have a literature of their own. As far as we know, none of this has been applied to a screenplay corpus.

III. Corpus Integrity and Exclusion Audit

A. Source and identity keys

The corpus links full screenplays from the Internet Movie Script Database (IMSDb), which several contributors compiled by hand, to IMDb metadata. Each of the $5{,}204$ joined records has a title, release year, decade label, IMDb identifier, runtime, an IMDb rating between 1.0 and 10.0, and a path to the script file. Ratings have mean 5.98, median 5.80 and standard deviation 1.44. When the archived pipeline ran, every text was on disk; the current snapshot ships only 356 of them. Searching the repository history brought back $1{,}186$ files: $1{,}177$ joined records and nine files the metadata never references. They come from one contiguous block of the rating-ordered file list, so although they span release years 1958–2025, their ratings all lie between 5.0 and 5.7. Any analysis that needs raw text is limited to this subset.

We check four identity keys, from strictest to loosest. The *content digest* SHA256(b) of a file with bytes b catches the same script re-uploaded under a new name, which no metadata key would notice; when two files collide we keep whichever loaded first. The *IMDb identifier* is the canonical film key. *Title*$\times$*year* is a fallback for records whose identifier is missing or malformed. The *title family* is the loosest key and the one that matters for splitting. To build it we case-fold the title t, strip non-alphanumeric characters, drop a leading article, and keep deleting trailing sequel markers (an Arabic numeral, a Roman numeral up to X, or `part` n) until nothing changes. *Toy Story* and *Toy Story 2* both end up in the family `toy story`, the grouping variable of Section IV-D.

B. The exclusion flow

Let $\mathcal{R}_0$ be the joined record set, $|\mathcal{R}_0| = 5{,}204$. The pipeline is a composition of four filters applied in a fixed order, each defined by a predicate on a record r:

$$\pi_{\text{len}}(r) \equiv |\text{text}(r)| \geq 1000 \text{ characters}, \tag{1}$$

$$\pi_{\text{hash}}(r) \equiv \text{SHA256}\big(\text{bytes}(r)\big) \notin \mathcal{H}_{<r}, \tag{2}$$

$$\pi_{\text{year}}(r) \equiv \text{yr}(r) \in [1900,\, 2025], \tag{3}$$

$$\pi_{\text{id}}(r) \equiv \text{positional identity of } r \text{ is unique}, \tag{4}$$

where $\mathcal{H}_{<r}$ is the set of digests already seen in load order. The surviving sets are nested, $\mathcal{R}_k = \{\, r \in \mathcal{R}_{k-1} : \pi_k(r) \,\}$, and the realised cardinality chain on this corpus is

$$5{,}204 \xrightarrow{\pi_{\text{len}}} 5{,}195 \xrightarrow{\pi_{\text{hash}}} 5{,}195 \xrightarrow{\pi_{\text{year}}} 5{,}191 \xrightarrow{\pi_{\text{id}}} 5{,}164. \tag{5}$$

On the $1{,}186$ recovered files the digest finds two duplicate groups; in both, the second copy is one of the nine unreferenced files, so no joined record is lost. The check covers $1{,}177$ of the $5{,}204$ records (22.6%). The metadata keys can be computed for every record; they bound the remaining duplication but cannot rule it out. The length filter π_{len} was applied when the archived pipeline ran, while every text was still available.

TABLE I
Corpus integrity audit and exclusion ledger. Percentages are of the $5{,}204$ joined records.

Check / exclusion	Count	%
Joined records (IMSDb $\bowtie$ IMDb)	5,204	100.00
Distinct IMDb identifiers	5,204	100.00
Duplicate title$\times$year pairs	0	0.00
Title collisions (same title, other year)	80 / 165 rec.	3.17
SHA-256 duplicate digests (1,177 rec.)	0	—
Script text $< 1{,}000$ characters	9	0.17
Release year outside $[1900, 2025]$	4	0.08
Unresolvable positional identity	27	0.52
Analysis corpus	**5,164**	**99.23**
Title families / multi-member	4,979 / 164	—

C. Audit results

Table I gives the complete ledger for the flow in Section III-B. The $5{,}204$ identifiers are all distinct and no title$\times$year pair occurs twice, so the join itself is clean and leakage here does not come from plain duplicate records. Eighty titles occur more than once (165 records), but each collision involves different identifiers and different years: these are remakes, or unrelated films that share a title, not copies, so they stay in the corpus as one family. Four records list release years of 1080 or 1088, obvious typos for 1980 and 1988; we remove them instead of guessing a repair. The derived decade column is in worse shape. The shipped label encoder knows 14 decade strings, `1080s` and `1996s` among them, and turns them into an *ordinal* integer.

We exclude 27 more records for reproducibility reasons. The archived embedding matrix stores rows in loader order with no record key, so row i of the matrix need not be record i of the spreadsheet. We rebuilt the offset function from $1{,}559$ anchors in the committed split bookkeeping. Inside the gaps of that reconstruction, 27 positions fit more than one assignment, and we drop them rather than guess.

IV. Methodology and Representations

A. Structural screenplay statistics

Sixteen statistics are computed from the raw, uncleaned script text: six size measures (`char_count`, `word_count`,

line_count, sentence_count, scene_count, words_per_scene); three describing lexical sophistication (avg_word_length, unique_word_ratio, long_word_ratio); two describing sentence geometry (avg_sentence_length, sentence_length_std); two describing dialogue (dialogue_density, the share of lines that match an ALL-CAPS speaker-cue pattern, and unique_characters); two emotional proxies (exclamation_ratio, question_ratio); and one stage-direction proxy (action_density). Two of the sixteen are close to degenerate on this corpus. Mean dialogue_density is 0.0087 with standard deviation 0.0361, and mean unique_characters is 3.41. A feature film has far more than three speaking parts. The speaker-cue regular expression assumes a NAME: convention that most IMSDb transcripts do not follow, so both features encode transcription format rather than dramaturgy.

B. Token-aligned chunking and mean pooling

Let a screenplay be a word sequence $D = (w_1, \ldots, w_N)$ obtained by whitespace splitting after light normalisation that removes bracketed stage directions, ALL-CAPS speaker cues, INT./EXT. markers, and timestamps. With window length $L = 256$ and overlap $o = 50$, the stride is $s = L - o = 206$, and the j-th window is

$$C_j \;=\; \big(w_{(j-1)s+1}, \ldots, w_{\min\{(j-1)s+L,\, N\}}\big), \qquad (6)$$

for $j = 1, \ldots, M$ with $M = 1$ if $N \le L$ and $M = \lceil (N - L)/s \rceil + 1$ otherwise. Each window is encoded by $f : \mathcal{C} \to \mathbb{R}^d$ with $d = 384$, and the document representation is the arithmetic mean

$$e(D) \;=\; \frac{1}{M} \sum_{j=1}^{M} f(C_j). \qquad (7)$$

Because the encoder L2-normalises its output, every $f(C_j)$ lies on the unit sphere, so

$$\big\|e(D)\big\|_2^2 = \frac{1}{M^2}\Big(M + 2 \sum_{1 \le i < j \le M} \langle f(C_i), f(C_j) \rangle\Big), \qquad (8)$$

which equals 1 only when all windows embed identically and falls toward $1/\sqrt{M}$ as they decorrelate. With a median of 33 windows per recovered script, any single scene has little influence on the pooled vector.

The word/token mismatch. The encoder's transformer accepts $T_{\text{max}} = 256$ positions, two of which are consumed by [CLS] and [SEP]. The content budget is therefore $B = T_{\text{max}} - 2 = 254$ WordPiece tokens, while C_j is defined by a count of 256 *words*. Writing $\tau(\cdot)$ for the tokeniser, the encoder in fact sees $\tau(C_j)$ truncated to its first B entries, and the discarded fraction for a document is

$$\rho(D) \;=\; 1 - \frac{\sum_{j=1}^{M} \min\big\{|\tau(C_j)|,\, B\big\}}{\sum_{j=1}^{M} |\tau(C_j)|}. \qquad (9)$$

Running the encoder's own tokeniser over the pipeline's windows on the 1,186 recovered scripts yields 44,899 windows with median length 373 tokens, 90th percentile 409 and maximum 706. Of these, 98.4% exceed B, and ρ computed over all tokens is 0.319 (worst script 0.611). Roughly one word in three is written to a window position that the encoder never reads, and the pipeline reports nothing. The 356-file public slice gives almost the same figures (98.45% and 0.316), so the rate is stable across subsets, although the recovered files cover only ratings 5.0–5.7.

C. Lexical control and learners

The lexical control is TF-IDF over unigrams and bigrams, capped at 8,000 features, with English stop words removed, minimum document frequency 3, maximum document frequency 0.85, and sublinear term frequency $1+\log \text{tf}$. It uses the same gradient-boosted regressor as the semantic arm. In the archived runs, however, the TF-IDF arm received text alone, whereas the SBERT arm also received the sixteen structural statistics and the three metadata columns and was trained with inverse-frequency sample weights. Section V-C reports a like-for-like rerun.

The regressor throughout is gradient-boosted trees with 500 rounds, learning rate 0.05, maximum depth 6, histogram splitting, and early stopping after 20 rounds without validation improvement; predictions are clipped to $[1, 10]$. Two linear probes provide reference points: ordinary least squares on the three metadata columns, and ridge regression ($\alpha = 1.5$) on standardised embeddings. A constant-mean predictor anchors the bottom. For the stacked ensemble, a ridge meta-regressor is fitted over four base models — metadata OLS, structural OLS (sixteen statistics plus metadata), TF-IDF+XGBoost, SBERT+XGBoost — using inner five-fold out-of-fold meta-features nested within each outer fold. In Tables II–IV, SBERT + XGBoost uses the 384 embedding dimensions only, and SBERT + meta + XGBoost adds runtime, release year and the decade code (387 inputs).

D. Partition protocols

Let $\mathcal{I} = \{1, \ldots, n\}$ index the analysis corpus, $n = 5{,}164$.

P1, random. A shuffled K-fold partition, $K = 5$: disjoint $F_1, \ldots, F_K$ with $\bigcup_k F_k = \mathcal{I}$, assignment independent of every record attribute. Within each training pool, a random 18.75% slice is held out for early stopping; the same applies to P2.

P2, grouped. Let $\gamma : \mathcal{I} \to \mathcal{G}$ map each record to its title family, $|\mathcal{G}| = 4{,}979$. GroupKFold constructs disjoint folds subject to the group constraint

$$\forall g \in \mathcal{G} \;\; \exists!\, k \in \{1, \ldots, K\} : \; \gamma^{-1}(g) \subseteq F_k, \qquad (10)$$

so no family is split across the boundary. Because 4,979 families cover 5,164 records, the constraint is nearly free: realised fold sizes are 1,032–1,033, identical to P1.

P3, chronological. With $\text{yr}(i)$ the release year, the test set is $2020 \le \text{yr}(i) \le 2025$, validation $c \le \text{yr}(i) \le 2019$, and training $\text{yr}(i) < c$, where c is the 85th percentile of the pre-2020 year distribution, here $c = 2016$. Early stopping is thereby decided on films that are themselves in the future relative to the training set, so no forward-looking information

enters through model selection. The realised sizes are 3,918 / 731 / 515. All models, including the linear baselines, are fitted on the training years only.

A partition's leakage can be measured directly. Define the family-overlap rate of a test fold as the share of its records whose family also occurs in the corresponding training pool,

$$\lambda(F_k) = \frac{1}{|F_k|}\big|\{\, i \in F_k : \gamma(i) \in \gamma(\mathcal{I} \setminus F_k)\,\}\big|. \tag{11}$$

Averaged over folds, $\lambda = 5.50\%$ under P1 and, by construction, $\lambda = 0$ under P2. The chronological holdout sits at $\lambda = 6.41\%$ — a sequel released in 2021 may well have a parent film in the training years.

E. Inference

Point estimates carry percentile bootstrap intervals from $B = 1000$ resamples of the test set. Model comparison is paired and distribution-free. For predictions $\hat{y}^A$, $\hat{y}^B$ on a common test set, form per-record absolute errors $e_i^A = |\hat{y}_i^A - y_i|$ and e_i^B likewise, take $d_i = e_i^A - e_i^B$, discard the n_0 exact ties, rank the remaining $|d_i|$, and accumulate

$$R^+ = \sum_{i:\, d_i > 0} \operatorname{rank}(|d_i|), \qquad R^- = \sum_{i:\, d_i < 0} \operatorname{rank}(|d_i|), \tag{12}$$

with test statistic $W = \min(R^+, R^-)$. The signed-rank test is non-parametric and has no degrees-of-freedom parameter; the reporting quantities are the number of pairs n, the number of non-tied pairs $n - n_0$, the statistic W, and the matched-pairs rank-biserial effect size

$$r \;=\; \frac{R^+ - R^-}{R^+ + R^-} \;\in\; [-1, 1], \tag{13}$$

which we report alongside p so that a small effect at large n is not mistaken for a large one. Differences in mean absolute error carry their own paired bootstrap intervals.

V. Experimental Results

A. The cost of a stricter partition

Table II shows the cost of the group constraint and Table III the cost of the temporal one. The five systems, the corpus and the learner settings are the same throughout.

Grouping costs little. The best system moves from $R^2 = 0.5490$ to 0.5359, a drop of 0.0131, and the two bootstrap intervals overlap substantially. This is expected for a corpus in which only 164 families have more than one member and the largest has four. The direction of the change is nonetheless consistent: every system that reads text loses R^2 (SBERT ridge by 0.0125, SBERT + XGBoost by 0.0127), while the metadata-only probe does not move (0.3888 to 0.3889). Franchise leakage therefore enters through the text, which a metadata model never sees.

Chronology costs much more. The best system falls to $R^2 = 0.3089$, and its interval $[0.1985, 0.4038]$ no longer overlaps either in-corpus estimate. The constant-mean predictor drops from $R^2 \approx 0$ to -0.0947, a pure distribution-shift effect: the training mean (6.11, films up to 2015) is too high for the holdout (5.75). The metadata probe degrades most, from 0.3889 to 0.1473, losing 62% of its explained variance, while the text-only SBERT + XGBoost model loses 35% (0.4114 to 0.2671). That text-only figure, 0.267, is also the most relevant number for screenplay-stage use, because it relies on nothing that becomes known only after production.

B. Paired comparisons

Table IV reports every baseline against the best system on matched records.

The last row matters most. Under P1 and P2, adding the three metadata columns to the embedding lowers MAE by 0.108–0.114 rating points, with very small p-values. Under P3 the same addition lowers MAE by only 0.0236, the interval includes zero, and $p = 0.273$ ($W = 62{,}734$ over $n = 515$ pairs, no ties, rank-biserial $r = +0.056$). By the usual conventions that effect is negligible. Metadata that looks highly significant under random folds brings no measurable gain on future films.

C. The lexical and structural arms

The lexical arm and the structural model need raw text for the whole corpus, so they exist only as archived runs on the original random splits over 5,195 records: five folds, plus one $70/15/15$ split whose test predictions were saved. The archived structural OLS uses the sixteen statistics together with the three metadata columns; it reaches $R^2 = 0.440 \pm 0.007$ against 0.382 for metadata alone, so the statistics add about 0.06. The archived SBERT arm also received all nineteen numeric columns, whereas TF-IDF + XGBoost saw text only, so its margin over TF-IDF mixed representation with metadata. For a fair comparison we retrained an SBERT-only model on the archived embeddings with the TF-IDF arm's exact recipe (300 trees, no early stopping, the same sample weights, folds and seeds). Over the five folds, SBERT-only reaches $R^2 = 0.372 \pm 0.012$ against 0.433 ± 0.014 for TF-IDF (MAE 0.880 against 0.819). On the saved split, where errors can be paired over 780 test films, TF-IDF has lower MAE by 0.030 $[-0.010, 0.069]$ ($p = 0.088$). Mean-pooled sentence embeddings therefore do not outperform a bag-of-words model here; the archived advantage came from the metadata. In the original pipeline, inverse-frequency sample weighting hurts: removing it lowers MAE by 0.0482 $[0.039, 0.057]$ ($p = 2.5 \times 10^{-26}$). A nested ridge stack reaches $R^2 = 0.577 \pm 0.010$, improving on its strongest base by $\Delta\text{MAE} = 0.0134$ $[0.008, 0.019]$, and a 100-trial TPE search reaches $R^2 = 0.581$ $[0.547, 0.618]$ on a single held-out split. Both gains are small next to the 0.240 gap in R^2 between the random and chronological protocols.

D. Temporal shift in detail

The holdout target is compressed relative to the training years: standard deviation 1.17 against 1.45, mean 5.75 against 6.11. Part of the fall in R^2 is therefore arithmetic, since the same absolute error against a narrower target yields a lower R^2. MAE alone does not settle the matter: the best system's MAE is 0.7276 under P1 and 0.7232 under P3, but the constant predictor's MAE also falls, from 1.176 to 0.976. Relative to

TABLE II

GROUPED FIVE-FOLD RESULTS. P1 SHUFFLES RECORDS; P2 ENFORCES THE TITLE-FAMILY CONSTRAINT (10). POOLED OUT-OF-FOLD POINT ESTIMATES OVER 5,164 RECORDS. ΔR^2 IS THE EXPLAINED VARIANCE LOST TO THE CONSTRAINT, I.E. THE SHARE OF P1 SKILL THAT FRANCHISE OVERLAP WAS SUPPLYING.

	P1 random k-fold ($\lambda = 5.50\%$)			**P2 grouped by family** ($\lambda = 0\%$)			
System	RMSE	MAE	R^2	RMSE	MAE	R^2	ΔR^2
Constant mean	1.4373	1.1761	-0.0000	1.4375	1.1765	-0.0004	-0.0004
Metadata OLS	1.1237	0.8701	0.3888	1.1236	0.8695	0.3889	$+0.0001$
SBERT ridge	1.1122	0.8618	0.4012	1.1238	0.8742	0.3887	-0.0125
SBERT + XGBoost	1.0907	0.8355	0.4241	1.1026	0.8445	0.4114	-0.0127
SBERT + meta + XGBoost	**0.9652**	**0.7276**	**0.5490**	**0.9791**	**0.7309**	**0.5359**	-0.0131

Best system: P1 $R^2 \in [0.5298, 0.5671]$, P2 $R^2 \in [0.5162, 0.5537]$. Every arm that reads text loses R^2; the metadata-only probe does not.

TABLE III

TEMPORAL HOLDOUT. FIT ON FILMS RELEASED UP TO 2015, EARLY STOPPING ON 2016–2019, TESTED ON THE 515 FILMS RELEASED 2020–2025.

System	RMSE	MAE	R^2
Constant mean	1.2230	0.9762	-0.0947
Metadata OLS	1.0794	0.8384	0.1473
SBERT ridge	1.0719	0.8417	0.1591
SBERT + XGBoost	1.0007	0.7469	0.2671
SBERT + meta + XGBoost	**0.9717**	**0.7232**	**0.3089**

Best system $R^2 \in [0.1985, 0.4038]$; $\lambda = 6.41\%$.
Target mean $\pm$ SD: train ($\leq$ 2015) 6.11 ± 1.45; pre-2020 pool 6.01 ± 1.46; holdout 5.75 ± 1.17.

TABLE IV

PAIRED COMPARISON AGAINST SBERT + METADATA + XGBOOST. ΔMAE IS POSITIVE WHEN THE REFERENCE SYSTEM HAS LOWER ERROR. p FROM THE TWO-SIDED WILCOXON SIGNED-RANK TEST.

Prot.	**Baseline**	ΔMAE	**95% CI**	p
P1	Constant mean	$+0.4485$	$[+0.427, +0.471]$	2.2×10^{-281}
	Metadata OLS	$+0.1425$	$[+0.129, +0.157]$	1.8×10^{-73}
	SBERT ridge	$+0.1342$	$[+0.116, +0.150]$	7.0×10^{-55}
	SBERT + XGB	$+0.1079$	$[+0.093, +0.122]$	4.7×10^{-46}
P2	Constant mean	$+0.4456$	$[+0.424, +0.466]$	2.9×10^{-280}
	Metadata OLS	$+0.1386$	$[+0.124, +0.152]$	2.8×10^{-73}
	SBERT ridge	$+0.1433$	$[+0.127, +0.159]$	2.5×10^{-61}
	SBERT + XGB	$+0.1136$	$[+0.100, +0.127]$	2.1×10^{-50}
P3	Constant mean	$+0.2530$	$[+0.195, +0.313]$	5.7×10^{-15}
	Metadata OLS	$+0.1152$	$[+0.063, +0.166]$	2.0×10^{-4}
	SBERT ridge	$+0.1185$	$[+0.064, +0.176]$	1.3×10^{-4}
	SBERT + XGB	$+0.0236$	$[-0.020, +0.074]$	0.273

that baseline, the best system removes 38% of baseline MAE under P1 and only 26% under P3, so accuracy does degrade on future films. Regressing prediction on truth over the holdout gives slope 0.463 (s.e. 0.028): the model separates above-average from below-average films but underestimates how far they lie from the mean.

VI. WHAT THE FEATURES ARE DOING

A. In-sample gain is not out-of-sample use

Split gain here comes from the archived SBERT model that saw all nineteen numeric inputs (403 features in total, random split). Embedding dimensions account for 55.5% of the total gain, the metadata columns for 36.6% (`movie_length` 25.5%, `year` 11.0%, `decade_encoded` 0.06%) and the structural statistics for 8.0%. Looked at per input, the order flips. The strongest embedding dimension contributes 1.68% and a typical one 0.14%, while runtime on its own is worth three times as much as all sixteen structural statistics put together.

B. Block permutation on the temporal holdout

Gain is measured on training splits, so it keeps every in-sample advantage; permutation on the holdout does not. For the P3 SBERT + meta + XGBoost model we shuffled each feature block across the 515 holdout films, re-predicted, and recorded the rise in MAE over 20 repetitions (baseline MAE $=$ 0.7232):

- permuting all three metadata columns: $+0.2168 \pm 0.0250$;
- permuting `movie_length` alone: $+0.1988 \pm 0.0185$;
- permuting the entire 384-dimensional embedding: $+0.1950 \pm 0.0217$;
- permuting `year`: $+0.0000 \pm 0.0000$;
- permuting `decade_encoded`: $+0.0000 \pm 0.0000$.

Neither zero is a rounding artefact. All holdout films came out in 2020–2025 and the model only saw films up to 2015, so every holdout value of `year` falls past the largest split threshold in the training data. The trees send all of them down the same branch, and shuffling values that already land in the same leaf has no effect. A feature that carries 11% of in-sample gain is, at deployment, a constant.

Runtime behaves differently. A 130-minute film from 2023 sits in the same part of feature space as a 130-minute film from 1994, so runtime carries over across time, and by itself it is worth as much as the whole embedding block. But runtime belongs to the finished film and is unknown while a screenplay is assessed, so it cannot help screenplay-stage prediction.

VII. THREATS TO VALIDITY AND LIMITATIONS

Raw text survives for just 1,177 of the 5,204 records, all rated 5.0–5.7, so the hash audit and the truncation estimate rest on a narrow rating range, and TF-IDF and the structural statistics exist only as archived random-split runs. The embedding matrix cannot be rebuilt from the current snapshot either. IMSDb's selection bias cannot be corrected

after the fact: the mean rating drops from 7.87 for 1920s films to 5.48 for the 2010s, which says more about what people chose to upload than about the films. We evaluate on a single holdout window; rolling-origin evaluation would show how quickly accuracy decays. Title families miss remakes and spin-offs released under a different title, so P2 removes only part of the franchise overlap. Finally, we audit only mean-pooled `all-MiniLM-L6-v2`, and other encoders may truncate differently.

VIII. Conclusion

We set out to audit a screenplay-to-rating pipeline, not to improve it. At the record level the corpus is clean, but it holds 164 multi-member title families, four corrupt release years and a broken decade vocabulary that passes into the feature matrix untouched. The chunking setup silently throws away about 32% of WordPiece tokens, and when trained the same way the sentence embeddings do not beat TF-IDF. Above all, the headline metric depends more on the partition than on the model: $R^2 = 0.549$ with random folds, 0.536 with franchises kept together, 0.309 on unseen years and 0.267 from text alone. We recommend grouping by title family, holding out by release year, reporting baseline-relative error under target shift, aligning chunks with the tokeniser, and keeping an immutable record key next to every cached representation.

Reproducibility

Code, the archived embeddings and all result files are at https://github.com/efazulkarim/imdb-predictor/tree/sbert; every number regenerates from the committed JSON artifacts (seed 42). The screenplays are not redistributed, but IMDb identifiers and SHA-256 digests are enough to rebuild the corpus.

References

[1] J. Eliashberg, S. K. Hui, and Z. J. Zhang, "Assessing box office performance using movie scripts: A kernel-based approach," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 11, pp. 2639–2648, 2014.

[2] S. D. Hunter, S. Smith, and S. Singh, "Predicting box office from the screenplay: A text analytical approach," *J. Screenwriting*, vol. 7, no. 2, 2016.

[3] A. J. Reagan, L. Mitchell, D. Kiley, C. M. Danforth, and P. S. Dodds, "The emotional arcs of stories are dominated by six basic shapes," *EPJ Data Science*, vol. 5, no. 31, 2016.

[4] E. Chu and D. Roy, "Audio-visual sentiment analysis for learning emotional arcs in movies," in *Proc. IEEE ICDM*, 2017.

[5] A. Ramakrishna, V. R. Martínez, N. Malandrakis, K. Singla, and S. Narayanan, "Linguistic analysis of differences in portrayal of movie characters," in *Proc. ACL*, 2017.

[6] M. Shafaei, N. Safi Samghabadi, S. Kar, and T. Solorio, "Age suitability rating: Predicting the MPAA rating based on movie dialogues," in *Proc. LREC*, 2020, pp. 1327–1335.

[7] E. Mohamed and L. A. Ha, "A first dataset for film age appropriateness investigation," in *Proc. LREC*, 2020, pp. 1311–1317.

[8] Y. Zhang, M. Shafaei, F. González, and T. Solorio, "From none to severe: Predicting severity in movie scripts," in *Findings of EMNLP*, 2021, pp. 3951–3956.

[9] S. Kar, S. Maharjan, and T. Solorio, "Folksonomication: Predicting tags for movies from plot synopses using emotion flow encoded neural network," in *Proc. COLING*, 2018.

[10] M.-C. Chiu, T. Feng, X. Ren, and S. Narayanan, "Screenplay quality assessment: Can we predict who gets nominated?" in *Proc. 1st Joint Workshop Narrative Understanding, Storylines, and Events (NUSE)*, 2020.

[11] R. Sharda and D. Delen, "Predicting box-office success of motion pictures with neural networks," *Expert Systems with Applications*, vol. 30, no. 2, 2006.

[12] S. Asur and B. A. Huberman, "Predicting the future with social media," in *Proc. IEEE/WIC/ACM Web Intelligence*, 2010.

[13] A. Oghina, M. Breuss, M. Tsagkias, and M. de Rijke, "Predicting IMDb movie ratings using social media," in *Proc. ECIR*, 2012.

[14] G. Mishne and N. Glance, "Predicting movie sales from blogger sentiment," in *AAAI Spring Symposium*, 2006.

[15] M. Mestyán, T. Yasseri, and J. Kertész, "Early prediction of movie box office success based on Wikipedia activity big data," *PLoS ONE*, vol. 8, no. 8, 2013.

[16] C. T. Madongo, Z. Tang, and J. Hassan, "Movie box-office revenue prediction model by mining deep features from trailers using recurrent neural networks," *J. Adv. Inf. Technol.*, 2024.

[17] M. Z. Naeem, F. Rustam, A. Mehmood, I. Ashraf, and G. S. Choi, "Classification of movie reviews using term frequency-inverse document frequency and optimized machine learning algorithms," *PeerJ Comput. Sci.*, 2022.

[18] J. Tripathi, S. Tiwari, A. Saini, and S. Kumari, "Prediction of movie success based on machine learning and Twitter sentiment analysis using internet movie database data," *Indonesian J. Elect. Eng. Comput. Sci.*, vol. 29, no. 3, pp. 1750–1757, 2023.

[19] K. Lee, J. Park, I. Kim, and Y. Choi, "Predicting movie success with machine learning techniques: Ways to improve accuracy," *Inf. Syst. Front.*, vol. 20, pp. 577–588, 2018.

[20] W. R. Bristi, Z. Zaman, and N. Sultana, "Predicting IMDb rating of movies by machine learning techniques," in *Proc. ICCCNT*, 2019.

[21] A. L. Gomes, G. Vianna, T. Escovedo, and M. Kalinowski, "Predicting IMDb rating of TV series with deep learning: The case of Arrow," in *Proc. Brazilian Symp. Inf. Syst. (SBSI)*, 2022.

[22] V. Udandarao and P. Gupta, "Movie revenue prediction using machine learning models," *arXiv:2405.11651*, 2024.

[23] W. Xie, "Predicting movie success with multi-task learning: A hybrid framework combining GPT-based sentiment analysis and SIR propagation," *arXiv:2509.02809*, 2025.

[24] H. L. Vogel, *Entertainment Industry Economics*, 10th ed. Cambridge, U.K.: Cambridge Univ. Press, 2020.

[25] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019.

[26] I. Chalkidis, X. Dai, M. Fergadiotis, P. Malakasiotis, and D. Elliott, "An exploration of hierarchical attention transformers for efficient long document classification," *arXiv:2210.05529*, 2022.

[27] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv:2004.05150*, 2020.

[28] K. Gorman and S. Bedrick, "We need to talk about standard splits," in *Proc. ACL*, 2019.

[29] A. Søgaard, S. Ebert, J. Bastings, and K. Filippova, "We need to talk about random splits," in *Proc. EACL*, 2021.

[30] P. Lewis, P. Stenetorp, and S. Riedel, "Question and answer test-train overlap in open-domain question answering datasets," in *Proc. EACL*, 2021.

[31] A. Elangovan, J. He, and K. Verspoor, "Memorization vs. generalization: Quantifying data leakage in NLP performance evaluation," in *Proc. EACL*, 2021.

[32] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.

[33] A. Lazaridou *et al.*, "Mind the gap: Assessing temporal generalization in neural language models," in *Proc. NeurIPS*, 2021.

[34] E. M. Bender and B. Friedman, "Data statements for natural language processing," *TACL*, vol. 6, 2018.

[35] J. Dodge *et al.*, "Documenting large webtext corpora: A case study on the Colossal Clean Crawled Corpus," in *Proc. EMNLP*, 2021.